

Tap-it: Správa webovej aplikácie – databázová správa

1. Úvod

Tap-it je webová aplikácia na správu fitness centra, ktorá zabezpečuje odbavovanie pomocou QR kódov, overovanie členstva, sledovanie obsadenosti v reálnom čase a fakturáciu. Členovia si môžu prezerať stav svojho členstva, rezervovať hodiny a priestory a sledovať históriu vstupov. Správcovia fitness centra majú k dispozícii dedikovaný panel na správu používateľov, členstiev, noviniek a nastavení.

Tech Stack:

- Frontend: Next.js 15 (App Router), TypeScript, Tailwind CSS
- Backend: Next.js API routes a server components (Node.js runtime)
- Databáza: PostgreSQL cez Supabase (cloudový hosting)
- Autentifikácia: Supabase Auth (JWT relácie)
- Platby: Stripe (webhooky spracované na strane servera)

2. Integrácia databázy

Databáza: Supabase

Spôsob pripojenia: Aplikácia používa knižnice `@supabase/ssr` a `@supabase/supabase-js` na pripojenie k databáze. V závislosti od kontextu sa používajú tri typy klientov:

Browserový klient — používa sa v klientských React komponentoch pre operácie v reálnom čase alebo akcie spustené klientom:

[// src/lib/supabase/client.ts](#)

```
1 import { createBrowserClient } from "@supabase/ssr";
2
3 export function createClient() {
4   return createBrowserClient(
5     process.env.NEXT_PUBLIC_SUPABASE_URL!,
6     process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
7   );
8 }
9
```

Serverový klient — používa sa v server komponentoch a API routách. Relácie sú prenášané cez HTTP cookies, čím sa zabezpečuje zachovanie kontextu autentifikovaného používateľa na strane servera:

[// src/lib/supabase/server.ts](#)

```
1 import { createServerClient, type CookieOptions } from "@supabase/ssr";
2 import { cookies } from "next/headers";
3
4 export async function createClient() {
5   const cookieStore = await cookies();
6
7   return createServerClient(
8     process.env.NEXT_PUBLIC_SUPABASE_URL!,
9     process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
10    {
11      cookies: {
12        getAll() {
13          return cookieStore.getAll();
14        },
15        setAll(cookiesToSet: { name: string; value: string; options: CookieOptions }[]) {
16          try {
17            cookiesToSet.forEach(({ name, value, options }) =>
18              cookieStore.set(name, value, options)
19            );
20          } catch {
21            // setAll called from a Server Component - safe to ignore
22          }
23        },
24      },
25    }
26  );
27 }
```

Admin klient — používa sa výlučne v admin akciách na strane servera. Využíva kľúč servisnej role, ktorý obchádza Row-Level Security (RLS) a nikdy nie je vystavený klientovi:

[// src/lib/supabase/admin.ts](#)

```
1 export function createAdminClient() {
2   if (!adminClient) {
3     adminClient = createClient<AdminDatabase>(
4       requireEnv("NEXT_PUBLIC_SUPABASE_URL"),
5       requireEnv("SUPABASE_SERVICE_ROLE_KEY"),
6       {
7         auth: {
8           autoRefreshToken: false,
9           persistSession: false,
10        },
11      }
12    );
13  }
14
15  return adminClient;
16 }
```

Tok požiadavky: Akcia používateľa (napr. zobrazenie rezervácií) spustí renderovanie server komponentu → serverový klient sa inicializuje so session cookie používateľa → typovaný dotaz je odoslaný do Supabase cez HTTPS → PostgreSQL vyhodnotí dotaz a príslušné RLS politiky → výsledky sú vrátené ako JSON a zobrazené.

3. Bezpečnosť a izolácia dát

Autentifikácia je zabezpečená pomocou Supabase Auth. Po prihlásení je vydaný JWT token uložený v HTTP-only cookie. Každá ďalšia požiadavka obsahuje tento cookie, ktorý serverový klient používa na stanovenie kontextu autentifikovaného používateľa.

Ochrana routov pomocou middleware (src/middleware.ts) zachytáva každú požiadavku a overuje reláciu pred samotným vykreslením stránky:

[// src/middleware.ts](#)

```
1  const { data, error } = await supabase.auth.getUser();
2    user = data.user;
3
4    if (error?.code === "refresh_token_not_found") {
5      clearSupabaseCookies(request, response);
6      user = null;
7    }
```

Neautentifikovaní používatelia sú presmerovaní na prihlasovaciu stránku. Admin routy navyše overujú rolu používateľa v tabuľke profiles:

[// src/middleware.ts](#)

```
1  const { data: profile, error: profileError } = await supabase
2    .from("profiles")
3    .select("role")
4    .eq("id", user.id)
5    .maybeSingle();
6
7    if (profileError || !isAdminRole(profile?.role)) {
8      return NextResponse.redirect(new URL("/", request.url));
9    }
10
11    currentUserIsAdmin = true;
12  }
```

Filtrovanie podľa ID používateľa sa uplatňuje pri každom dotaze na strane používateľa, aby prihlásený používateľ nemohol získať dáta iného používateľa:

[// src/app/bookings/my/page.tsx](#)

```
1 const { data: bookings } = await supabase
2   .from("bookings")
3   .select("id, service_id, schedule_id, start_time, end_time, status, total_price, created_at, bookable_services(name, type)")
4   .eq("user_id", user.id)
5   .order("created_at", { ascending: false });
```

Row-Level Security (RLS) je vynútené na úrovni databázy prostredníctvom PostgreSQL politik definovaných v migračných súboroch. Aj keby bol obídený aplikačný kód, databáza by sama odmietla neoprávnený prístup. Napríklad tréneri môžu upravovať iba vlastné rozvrhy:

[-- supabase/migrations/20260519130000_monthly_trainer_availability.sql](#)

```
1 create policy "Trainers can manage their own service_schedules"
2   on public.service_schedules
3   for all
4   using (auth.uid() = trainer_id)
5   with check (auth.uid() = trainer_id);
```

Admin klient (servisná rola) je inicializovaný výlučne v súboroch serverových akcií, ktoré sú samy chránené kontrolou role v middleware, čím sa zabezpečuje, že rozšírené oprávnenia nie sú dostupné z prehliadača.

4. Príklady práce s databázou

Príklad 1 — Načítanie profilu používateľa

// src/app/settings/page.tsx

```
1  const { data: profile, error: profileError } = await supabase
2    .from("profiles")
3    .select("*")
4    .eq("id", user.id)
5    .single();
```

Dotaz SELECT na tabuľku profiles filtrovaný podľa ID autentifikovaného používateľa. Vracia jeden riadok s osobnými údajmi používateľa. Tento vzor zabezpečuje, že používatelia môžu čítať iba vlastný profil.

Príklad 2 — Počítanie aktívnej obsadenosti fitness centra

// src/app/page.tsx

```
1  const { count: openEntriesCount } = await supabase
2    .from("entries")
3    .select("id", { count: "exact", head: true })
4    .is("check_out", null)
5    .eq("is_valid", true);
```

Dotaz COUNT na tabuľku entries, ktorý počíta riadky, kde check_out je null (t. j. osoba sa odhlásila na vstupe, ale ešte neodišla). Používa sa na zobrazenie aktuálnej obsadenosti fitness centra na domovskej stránke.

Príklad 3 — Priradenie členstva administrátorom

// src/app/admin/(panel)/memberships/actions.ts

```
1  const { error: insertError } = await admin.from("user_memberships").insert({
2    user_id: userId,
3    membership_id: planRow.id,
4    start_date: nowIso,
5    end_date: getPlanDurationEndDate(planRow.duration_days),
6    entries_remaining: null,
7    status: "active",
8    activated_by_admin: true,
9  });
```

Operácia INSERT do tabuľky user_memberships vykonaná pomocou admin klienta (servisná rola). Vytvorí nový aktívny záznam členstva pre používateľa. Príznak activated_by_admin zaznamenáva, že akciu vykonal administrátor, nie samotný používateľ.

Príklad 4 — Zaznamenanie fakturačnej transakcie

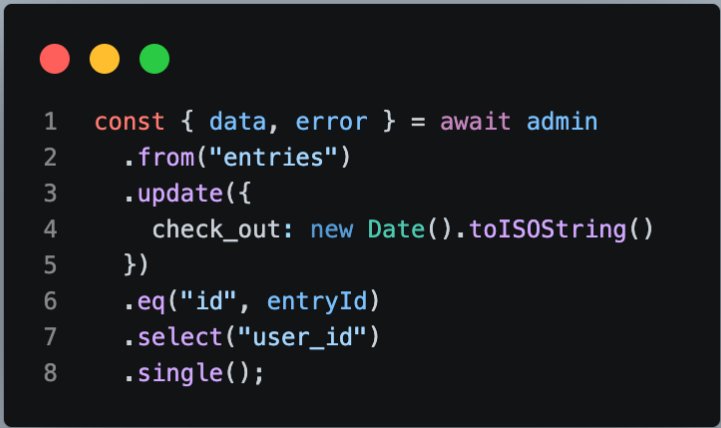
// src/app/admin/(panel)/memberships/actions.ts

```
1  const { error: purchaseTransactionError } = await admin
2    .from("transactions")
3    .insert({
4      user_id: userId,
5      membership_id: planRow.id,
6      amount: getTransactionAmount(planRow.price),
7      currency: "EUR",
8      type: "purchase",
9      status: "completed",
10     metadata: {
11       source: "admin_membership_change",
12       action: membershipPlan,
13       changed_by_admin_id: user.id,
14       old_membership_id: activeMembership?.membership_id ?? null,
15     },
16   });
```

Operácia INSERT do tabuľky transactions, ktorá zaznamenáva každú zmenu členstva ako finančný záznam. Stĺpec metadata typu JSONB uchováva auditné informácie vrátane toho, ktorý administrátor zmenu inicioval.

Príklad 5 — Odhlásenie používateľa administrátorom

// src/app/admin/(panel)/users/[id]/actions.ts



```
1  const { data, error } = await admin
2    .from("entries")
3    .update({
4      check_out: new Date().toISOString()
5    })
6    .eq("id", entryId)
7    .select("user_id")
8    .single();
```

Operácia UPDATE na tabuľke entries, ktorá nastaví časovú pečiatku check_out pre konkrétny záznam vstupu, čím označí odchod používateľa z fitness centra. Dotaz vracia user_id dotknutého záznamu, aby mohla nadväzujúca logika aktualizovať súvisiaci stav.

5. Záver

Tap-it používa databázu PostgreSQL spravovanú službou Supabase ako jediné úložisko dát. Aplikácia komunikuje s databázou výlučne cez HTTPS pomocou SDK @supabase/ssr a @supabase/supabase-js, pričom využíva tri úrovne klientov: browserový (verejný kľúč, klientska strana), serverový (relácia viazaná na cookie, serverová strana) a admin (servisná rola, výlučne serverová strana). Dáta sa načítavajú a ukladajú prostredníctvom typovaného query builderu, ktorý priamo mapuje na databázové tabuľky a pokrýva všetky štandardné CRUD operácie, relácie aj agregátne počty.

Bezpečnosť je vynútená na dvoch úrovniach. Na aplikačnej úrovni Next.js middleware overuje každú reláciu a kontroluje priradenie rolí pred udelením prístupu k chráneným routom. Všetky dotazy na strane používateľa navyše filtrujú podľa ID autentifikovaného používateľa, čím zabraňujú horizontálnemu úniku dát. Na úrovni databázy politiky Row-Level Security zaručujú dodržiavanie týchto obmedzení aj v prípade obídenia aplikačnej vrstvy, čím sa izolácia dát stáva štrukturálnou zárukou, nie len konvenciou.